



Practica 6

[Explicación de práctica 6.pdf](#)

Ejercicio 1:

a- Explique brevemente los siguientes conceptos

- **Parámetro**

-Variable que se define en la declaración de una función/método y sirve para recibir valores cuando esa función o método es llamada/o

- **Parámetro real**

-Valor concreto que se pasa a la función cuando se la invoca

- **Parámetro formal**

-Es el nombre de la variable que aparece en la definición de la función. Se vincula a los parámetros reales cuando la función se ejecuta

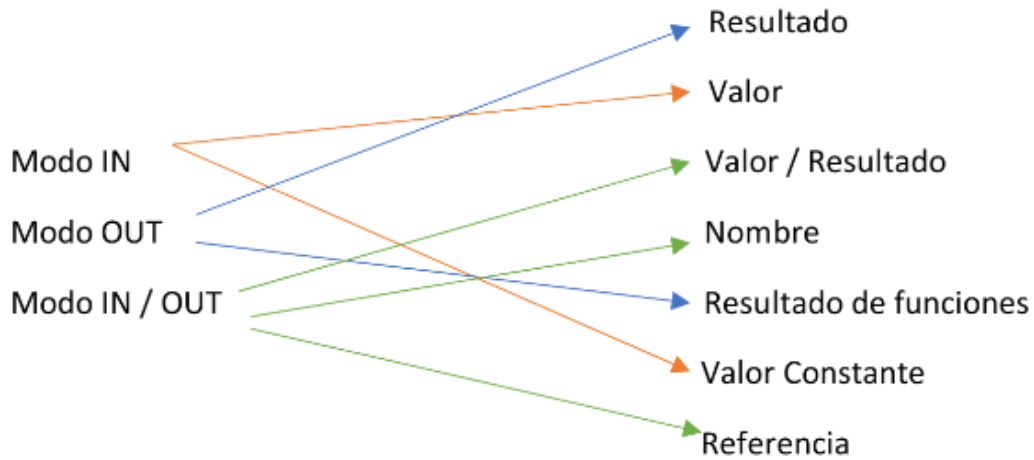
- **Ligadura posicional**

-Método para asociar parámetros reales con parámetros formales basado en el orden en el que se pasan los argumentos

- **Ligadura por palabra clave o nombre**

-Método para asociar parámetros reales con parámetros formales basado explícitamente por su nombre independientemente del orden

Ejercicio 2: Unir los siguientes puntos según corresponda y de una definición y un ejemplo de cada par.



-Modo IN: Parametro formal recibe el dato desde el parametro real

-Valor → El parametro formal recibe una copia del valor del parametro real, por lo que cualquier cambio dentro de la funcion no se ve reflejado en la variable original

-Valor constante → Parecido a valor solo que el parametro formal se trata como una constante dentro de la funcion y no se puede modificar

-Modo OUT: Parametro formal envia el dato al parametro real

-Por resultado → El parametro formal actua como una variable local dentro de la funcion y cuando termina esta misma su valor se copia al parametro real

-Por resultado de funciones → La funcion devuelve un valor a traves de su nombre o mecanismo de retorno y no necesariamente mediante parametros

-Modo IN/OUT

-Valor-Resultado → El parametro formal recibe una copia del valor inicial del argumento y al finalizar la funcion el valor final del parametro formal se copia de vuelta al argumento original

-Referencia → El parametro formal es un alias del parametro real (se pasa la direccion de memoria) y cualquier cambio sobre el parametro afecta directamente al argumento original

-Nombre → El parametro formal actua como una referencia simbolica que se evalua cada vez que se usa dentro de la funcion

Ejercicio 3:

a- Complete el siguiente cuadro según lo correspondiente a cada lenguaje:

Tipo de pasaje de parámetros	Lenguaje
Por valor (IN), Por resultado (OUT), Por Valor-Resultado (IN/OUT), por referencia	ADA
Por valor, Por referencia	C
Por referencia a objetos (unica forma)	Ruby
Por valor (unica forma con copoias de referencias para objetos)	JAVA
Por valor para tipos inmutables (int,str,etc) y Por refrencia para tipos mutables (listas, diccionarios)	Python

b- Ada es más seguro que Pascal, respecto al pasaje de parámetros en las funciones. Explique por qué.

-Esto es asi debido a que ADA ofrece una semantica explicita y estricta al momento del pasaje de parametros, por otro lado, Pascal depende mas del programdor y no brinda la garantia de verificacion en tiempo de compilacion

c- Explique cómo maneja Ada los tipos de parámetros in-out de acuerdo al tipo de dato

Esto se lo robo a guaymas pq no tengo idea donde esta

1. Tipos escalares (Integer, Float, Boolean, etc.)

Mecanismo: Por copia (valor-resultado).

Funcionamiento:

- El valor inicial del argumento se copia al parámetro formal al entrar al procedimiento.
- Dentro del procedimiento, se trabaja con esa copia.
- Al salir, el valor final del parámetro se copia de vuelta al argumento original.

2. Tipos compuestos pequeños (Registros o estructuras pequeñas sin restricciones especiales)

Mecanismo: Por copia (valor-resultado), generalmente.

Funcionamiento:

- Similar a los tipos escalares: el registro completo se copia al entrar y se copia de vuelta al salir.
- Se asegura consistencia y evita problemas de aliasing.

3. Tipos compuestos grandes o complejos (Arreglos, Registros grandes, Tipos con restricciones)

Mecanismo: Por referencia (implícito), aunque la semántica sigue siendo como valor-resultado desde la perspectiva del programador.

Funcionamiento:

- En lugar de copiar todo el objeto (lo cual sería ineficiente para estructuras grandes como arreglos), el compilador pasa una referencia al objeto original.
- Las modificaciones se realizan directamente sobre el objeto en memoria.

4. Tipos con restricciones (Constrained Types)

Mecanismo: Por copia o por referencia, dependiendo del tamaño y la implementación.

Funcionamiento:

- Si el tipo tiene restricciones (como un subtipo con un rango específico), Ada garantiza que las modificaciones respeten esas restricciones.
- Para tipos pequeños, se usa copia; para tipos grandes, se usa referencia.

5. Tipos protegidos o tareas (Protected Types, Tasks)

Mecanismo: Por referencia.

Funcionamiento:

- Los tipos protegidos o tareas son complejos y no se copian. Se pasan por referencia para mantener la integridad del objeto y permitir la sincronización.

Ejercicio 4: Sea el siguiente programa escrito en Pascal-like

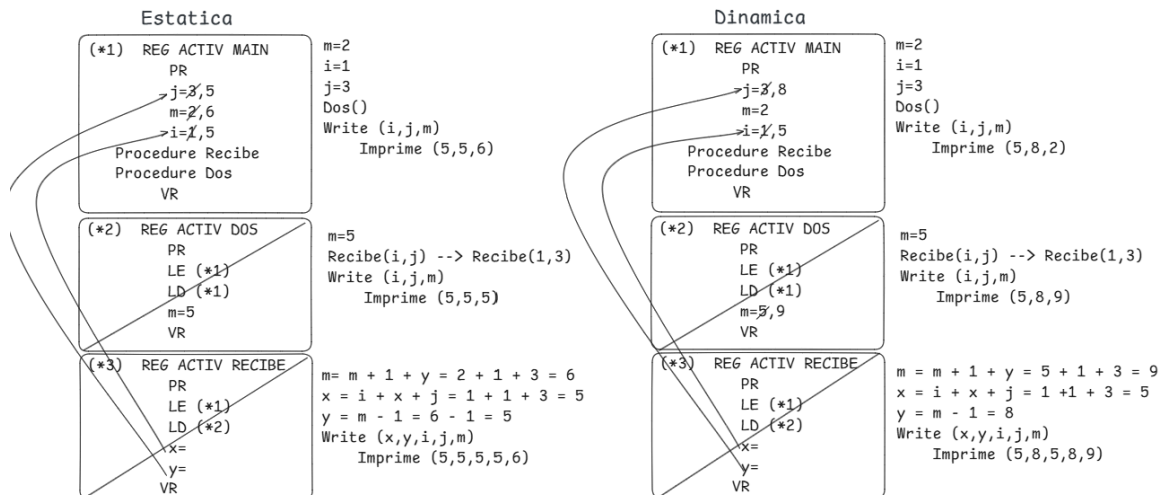
<pre>Procedure Main; var j, m, i: integer; Procedure Recibe (x:integer; y:integer); begin m:= m + 1 + y; x:=i + x + j; y:=m - 1; write (x, y, i, j, m); end;</pre>	<pre>Procedure Dos; var m:integer; begin m:= 5; Recibe(i, j); write (i, j, m); end; begin m:= 2; i:=1; j:=3; Dos; write (i, j, m); end.</pre>
---	--

a- Arme el árbol de anidamiento sintáctico y el registro de activación de cada una de las unidades.

b- Decir qué imprime el programa suponiendo que para todas las variables que se pasan el pasaje de parámetros es por: (Deberá hacer la pila estática y dinámica para cada caso)

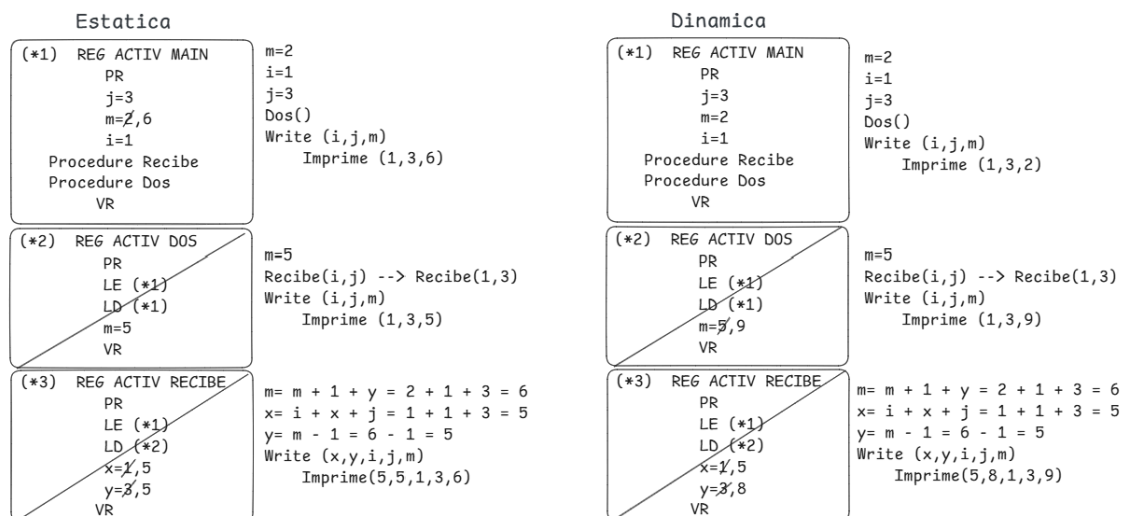
i- Referencia.

i) Referencia



ii- Valor

ii) Valor



iii-Valor Resultado

iii) Valor-Resultado

Estatica	Dinamica
<pre> (*1) REG ACTIV MAIN PR j=2,5 m=2,6 i=1,5 Procedure Recibe Procedure Dos VR </pre>	<pre> (*1) REG ACTIV MAIN PR j=2,8 m=2 i=1,5 Dos() Write (i,j,m) Imprime (5,5,6) </pre>
<pre> (*2) REG ACTIV DOS PR LE (*1) LD (*1) m=5 VR </pre>	<pre> (*2) REG ACTIV DOS PR LE (*1) LD (*1) m=5,9 VR </pre>
<pre> (*3) REG ACTIV RECIBE PR LE (*1) LD (*2) x=1,5 y=3,5 VR </pre>	<pre> (*3) REG ACTIV RECIBE PR LE (*1) LD (*2) x=1,5 y=3,8 VR </pre>

iv- Nombre

iv) Nombre

Estatica	Dinamica
<pre> (*1) REG ACTIV MAIN PR j=2,5 m=2,6 i=1,5 Procedure Recibe Procedure Dos VR </pre>	<pre> (*1) REG ACTIV MAIN PR j=2,8 m=2,6 i=1,5 Procedure Recibe Procedure Dos VR </pre>
<pre> (*2) REG ACTIV DOS PR LE (*1) LD (*1) m=5 VR </pre>	<pre> (*2) REG ACTIV DOS PR LE (*1) LD (*1) m=5,9 VR </pre>
<pre> (*3) REG ACTIV RECIBE PR LE (*1) LD (*2) x=i(*1) y=j(*1) VR </pre>	<pre> (*3) REG ACTIV RECIBE PR LE (*1) LD (*2) x=i(*1) y=j(*1) VR </pre>

v-Resultado.

-No se puede

c- ¿Existió algún caso que no pudo realizarlo porque saltó algún tipo de error? Diga cuál y por qué.

-Si en el inciso v en el pasaje de parametro por resultado genera error la sentencia $m=m+1+y$ porque el parametro y es pasado por resultado y nunca se inicializa dentro de la funcion

d- ¿Dará el mismo resultado si se trata de un lenguaje que sigue la cadena

dinámica? Justifique la respuesta realizando las pilas de activación

-No

Ejercicio 5: Suponiendo que se está ejecutando un programa con el siguiente registro de

activación en memoria y se llama al procedimiento rutina(iter,vec,a).

Determine el tipo de

parámetro que se deben utilizar en el llamado para que los resultados sean los siguientes:

a) (4,6,7), (4,6,7), 2, 2

-Modo IN/OUT por Referencia en todos los parametros

b) (3,5,6), (4,6,7), 2, 2

-Modo IN por Valor para el parametro vector, y Modo IN/OUT por Referencia en los otros 2

c) (3,5,6), (5,5,6), 0, -1

-Modo IN por Valor o IN/OUT por Valor-Resultado para todos los parametros

PR
LD
LE
Iter: true
Vec:[3,5,6]
a: -1
Rutina()
VR

```
.....
procedura rutina(tipoParam iteracion,tipoParam vector,tipoParam vit):

    while iteracion begin
        vit = a+1
        vector[vit] = vector[vit]+1
        iteracion = (vector[vit] mod 2)==0
    end
    print vec
    print vector
    print vit
    print a
.....

rutina(iter,vec,a)
```

Ejercicio 6: Indique con un ejemplo el comportamiento del parámetro por nombre (en el parámetro formal) para los siguientes casos de parámetros reales:

- Un valor entero.
- Una constante.

- Un elemento de un arreglo.
- una expresión.

Que sucede en cada caso?

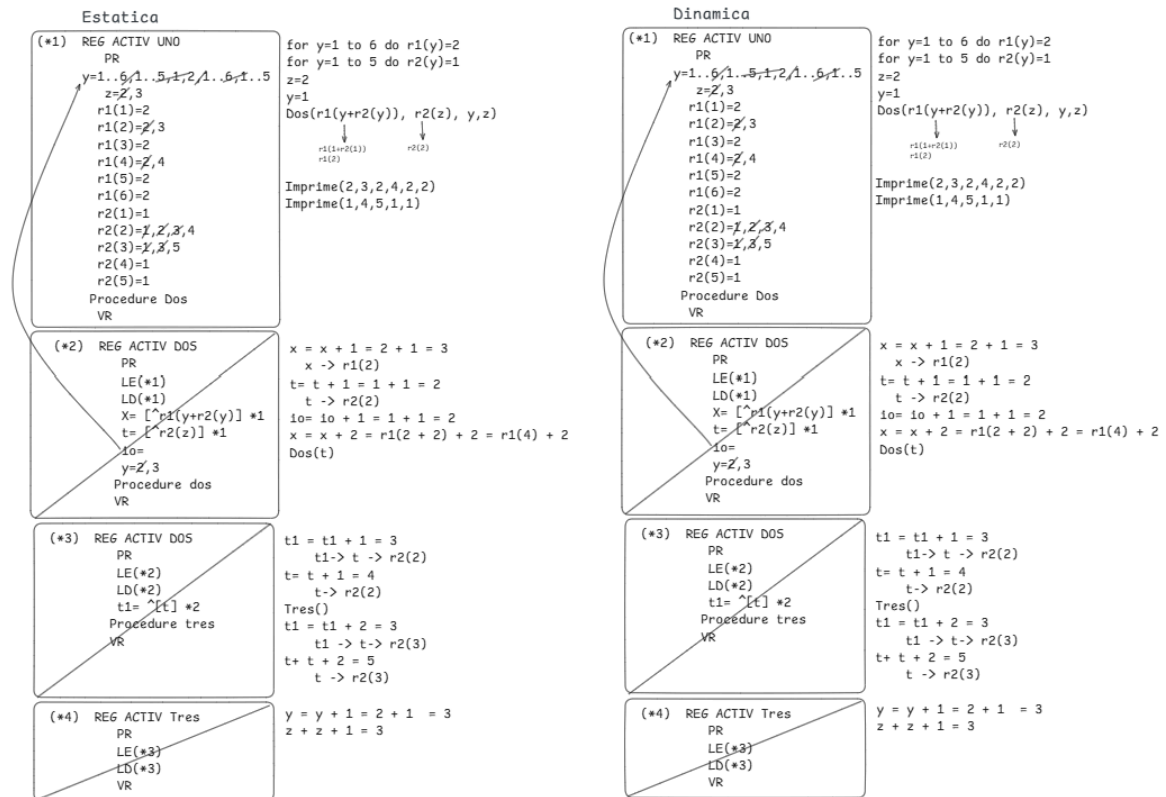
Si el dato a compartir es:

- Un único valor se comporta exactamente igual que el pasaje por referencia.
- Si es una constante es equivalente a por valor.
- Si es un elemento de un arreglo puede cambiar el suscripto entre las distintas referencias
- Si es una expresión se evalúa cada vez

Ejercicio 7: Realice la pila de ejecución del siguiente programa:

<pre> Procedure Uno; y, z: integer; r1:array[1..6] of integer; r2:array[1..5] of integer; Procedure Dos(nombre x, t:integer; var io:integer; valor-resultado y:integer); Procedure Dos(nombre t1:integer); Procedure Tres; begin y:= y + 1; z:= z + 1; end; begin t1:= t1 + 1; t:= t + 1; Tres; t1:= t1 + 2; t:= t + 2; end; </pre>	<pre> begin x:= x + 1; t:= t + 1; io:= io + 1; x:= x + 2; if z =2 then Dos (t); end; begin for y:= 1 to 6 do r1(y):= 2; for y:= 1 to 5 do r2(y):= 1; z:= 2; y:= 1; Dos(r1(y + r2(y)), r2(z), y, z); for y:= 1 to 6 do write (r1(y)); for y:= 1 to 5 do write (r2(y)); end. </pre>
---	--

a) siguiendo la cadena estática b) siguiendo la cadena dinámica



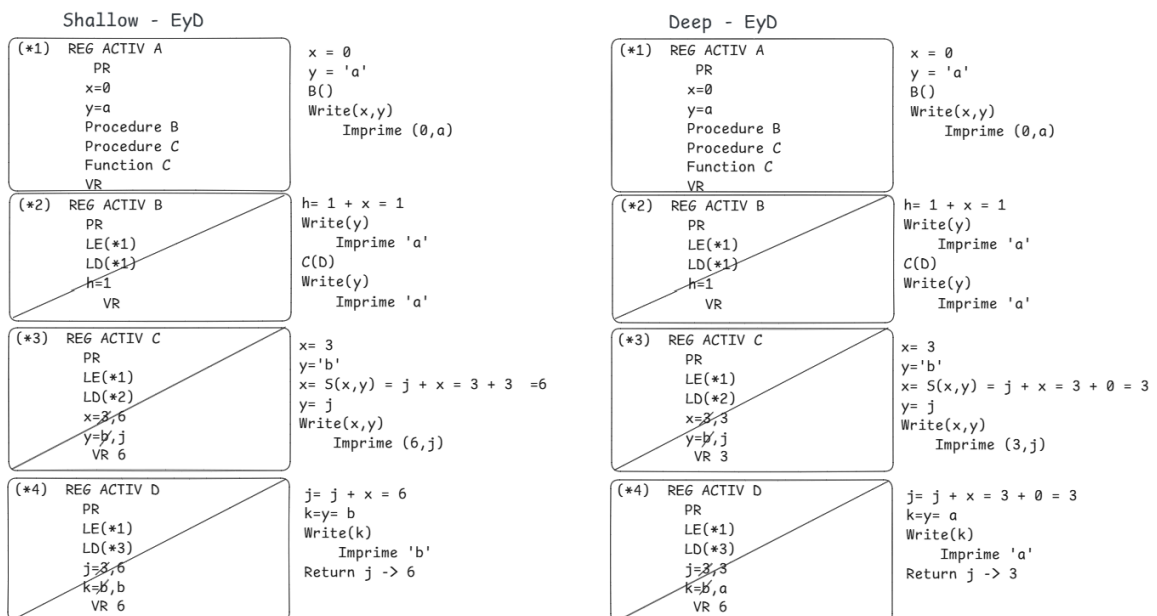
Ejercicio 8:

a) Indique las diferencias entre los pasaje de subprogramas como parámetros deep y shallow.

- Ligadura shallow → El ambiente de referencia es el del subprograma que tiene declarado el parametro formal del subprograma
- Ligadura deep → El ambiente es el del subprograma donde esta declarado el subprograma usado como parametro real.

b) Realice la pila estática y dinámica tanto con el pasaje de parámetros deep y shallow para el siguiente código.

<pre> Program A Var x:integer; Var y: char; Procedure B; Var h:integer; Begin h:=1+x; Write (y); C(D); Write (y); End; Procedure C (Subrutina S); Var x:integer; Var y: char; Begin x:=3; y:= "b"; x:=S(x,y) y:= "j"; Write (x,y); End; </pre>	<pre> Function D (j:integer, k:char); Begin j:=j+x; k:=y; Write (k); Return j; End; BEGIN x:=0; y:="a"; B(); Write (x,y); END. </pre>
--	---



Ejercicio 9: Sea el siguiente código escrito en Pascal like

<pre> Procedure main a: array(1..5) of integer; x: integer; i:integer; Procedure Uno (tipo_pasaje m:integer) Begin x:=0; x:=x+1; m:=m+x + a(3); x:=x*2; a(3):=a(3) - 1; m:=m+1; End; </pre>	<pre> Begin For i:=1 to 5 a(i):=1; x:=3; Uno(a(x)); For i:=1 to 5 write (a(i)); End. </pre>
---	---

a- Plantee diferencias, relacionada con la forma de implementación de cada uno y los resultados sobre este ejemplo considerando los siguientes tipos de pasajes parámetros nombre, referencia y valor resultado.

-Nombre: El parametro formal es sustituido textualmente por una expresion del parametro real mas un puntero al entorno del parametro real. Se establece la ligadura entre parametro formal y real en el momento de la invocacion pero la ligadura de valor se difiera hasta que se lo utiliza (direccion se resuelve en ejecucion). No apunta a una direccion fija y puede ir cambiando

-Referencia: Parametro formal es una variable local que contiene la direccion al parametro real de la unidad llamadora que esta en un ambiente no local. Cualquier cambio sobre el parametro formal se vera reflejado en el parametro real

-Valor-Resultado: Parametro formal es una variable local que recibe una copia del contenido (a la entrada) del parametro real y el parametro real (a la salida) recibe una copia de lo que tiene el parametro formal. Basicamente lo que hace la rutina lo copio en la variable

TREMENDA PAJA HACER POR 5TA VEZ LA PILA DE EJECUCION

a) nombre

x=1

m

$a(x) = a(x) + x + a(3)$

$a(1) = a(1) + 1 + a(3) = 3$

x=2

$a(3) = a(3) - 1 = 0$

m

$a(x) = a(x) + 1$

$a(2) = a(2) + 1$

referencia

x=1

m

$a(3) = a(3) + 1 + a(3) = 3$

$a(3) = a(3) - 1 = 2$

m

$a(3) = a(3) + 1$

valor resultado

x=1

$m = m + x + a(3) = 3$

$m = m + 1 = 4$

	3	2	0			
a	1	1	1	1	1	1

imprime 3, 2, 0, 1, 1

			3			
a	1	1	1	1	1	1

imprime 1, 1, 3, 1, 1

			4			
a	1	1	1	1	1	1

imprime 1, 1, 4, 1, 1

b- ¿Qué sucede si en Uno se agrega la siguiente declaración: x: integer?
Indique el resultado para cada uno de los tipos de pasajes de parámetros (nombre, referencia y valor resultado)

-Cambia que los valores a imprimir son 1,1,3,1,1 en el caso de nombre y referencia y 1,1,4,1,1 en caso de valor-resultado

Ejercicio 10:

<pre> Program Uno; var x:integer; Function Dos:integer; begin x:= x + 1; return (x); end; </pre>	<pre> Procedure Tres (pasaje x:integer); begin x:= x + 5; x:= Dos + 10; end; begin x:= 8; Tres(x); write (x); end. </pre>
--	--

a- Explique cómo simularía en Pascal el pasaje por valor-resultado y hágalo sobre este ejemplo.

Nota: No se pueden agregar más variables, ni cambiar el nombre de las que están.

```
Program Uno;
var x: integer;
Function Dos:Integer;
begin
    x:= x + 1;
    return (x)
end;
Function Tres(x:integer):integer
begin
    x:= x + 5;
    x:= Dos + 10;
    Tres:= x
end;

begin
    x:= 8;
    x:= Tres(x); // Llamo a Tres y me va a devolver el r
                // Dos + 10 y queda guar
resultado de la funcion      dado en X, queda como valor-resultado
                                // ya que se copia el 8
                                // lo retorna
    write(x);
end.
```

b- Transcriba este ejemplo en Ada de manera tal que el resultado de la ejecución sea diferente si el pasaje de parámetros es por referencia y luego por valor - resultado